ARTI 403 — Image Processing

Lab 5 — Spatial Filtering: Smoothing and Sharpening

Student name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1
College	Computer Science and Information Technology
Department	Computer Engineering
University	Imam Abdulrahman Bin Faisal University
Date	13 October 2026

1. Objective

Outcome #2: Write a program that implements fundamental image processing algorithms.

This lab covers spatial filtering: operations in which the output at a pixel depends on its **neighbourhood** rather than on that pixel alone. Smoothing with box and Gaussian filters, and sharpening with unsharp masking and the Laplacian, are all the same mechanism with different weights.

2. Tools

Component	Detail
Python	3 (Anaconda)
IDE	Jupyter Notebook (via Visual Studio Code)
Libraries	OpenCV, NumPy, Pillow, scikit-image, Matplotlib
Image	images/Parrot.png — a colour photograph

3. Background — what a spatial filter is

A small grid of weights, the **kernel**, is slid over the image. At each position the pixels under the kernel are multiplied by the corresponding weights and summed, and that single number becomes the output pixel. Every filter in this lab is this one operation with a different set of weights.

The **sum of the weights** predicts the behaviour, and the two cases used here are:

Sum of weights	Consequence	Filters
1	brightness preserved; the filter averages	box, Gaussian
0	zero output on flat regions; the filter detects change	Laplacian

A kernel summing to 1 returns the same value on a uniform region, so the overall brightness cannot drift. A kernel summing to 0 returns nothing on a uniform region, which is exactly what makes the Laplacian a detector rather than a filter: it reacts only where the intensity changes.

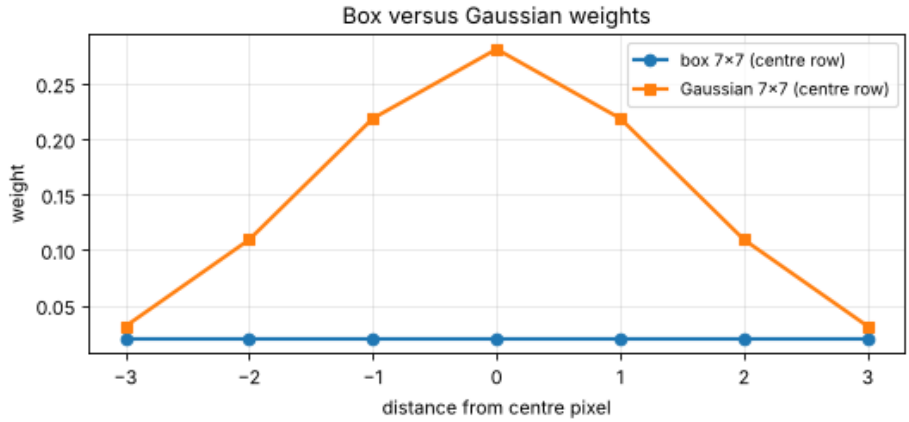


Figure 1 — the centre row of each smoothing kernel. The box filter gives a neighbour three pixels away the same influence as the centre pixel; the Gaussian tapers with distance.

4. Unsharp masking

Unsharp masking sharpens an image by **blurring** it first, which is counter-intuitive until the three steps are laid out:

1. Blur the image — the blurred copy keeps only the broad, slowly-varying content.
2. Subtract it from the original — what remains is exactly the detail the blur removed. This difference is the **mask**.
3. Add the mask back, amplified — the detail is exaggerated, and the eye reads that as sharpness.

```
g = f + k (f - blur(f))
```



Figure 2 — original, blurred and sharpened, with sigma = 2 and amount = 1.5.

4.1 Why the image must be converted to float first

`image.astype(np.float32) / 255.0` is not cosmetic. The mask `image_float - blurred` ranges from -0.5157 to 0.7891, and **53.6% of its values are negative** — one side of every edge. In `uint8` those negatives would wrap round to large positive numbers and ruin the result. Working in float preserves them, and a single `np.clip(..., 0, 1)` at the end folds the range back.

Before clipping, the amplified result spans -0.713 to 2.176: 4.6% of values fall below 0 and 2.3% rise above 1.

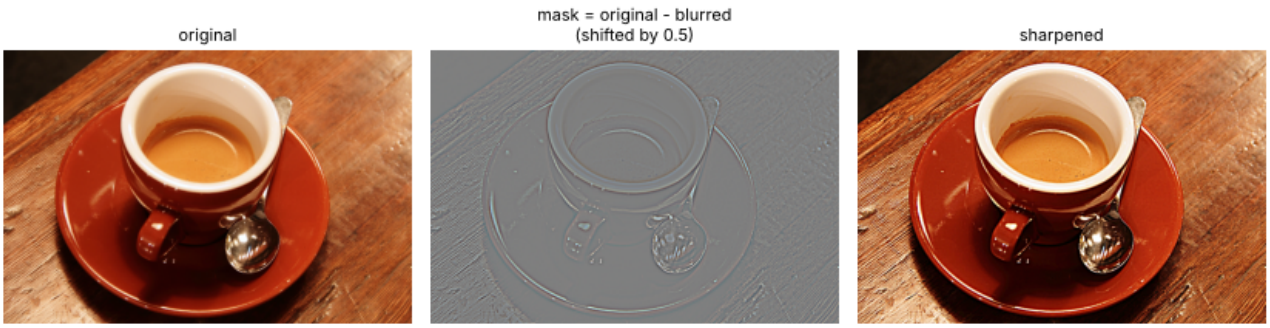


Figure 3 — the mask itself, shifted by 0.5 so the negative values are visible. It is grey where the image is flat and bright or dark along edges.

4.2 The effect of the amount

`sigma` decides which detail counts as detail; `amount` decides how hard it is pushed back in. Too large a value produces **halos** — bright and dark outlines flanking strong edges, the standard sign of over-sharpening.



Figure 4 — increasing the amount on a crop. Halos become obvious at the highest value.

5. Task 1 — Convolution with a 7x7 box filter

A box filter replaces each pixel with the plain average of the 7x7 block around it. The kernel was built explicitly — 49 identical weights of 1/49 — and applied with `cv2.filter2D`, the general convolution function.



Figure 5 — original and the result of the 7x7 box filter.

5.1 Verification

`cv2.blur` is OpenCV's dedicated box-filter call, so if the kernel was built correctly the two results must match exactly. They do: the largest difference is 0 intensity levels.

Quantity	Before	After	Interpretation
Mean brightness	98.62	98.61	preserved — the weights sum to 1
Standard deviation	74.08	71.75	reduced — detail is averaged away

Kernel size	Standard deviation	Mean brightness
1x1 (original)	74.08	98.62
3x3	72.95	98.61
7x7	71.75	98.61
15x15	69.84	98.62
31x31	66.8	98.62



Figure 6 — the box filter at increasing kernel sizes. The mean holds steady while the standard deviation falls.

A note on terminology. `cv2.filter2D` performs *correlation*, not true convolution: it does not flip the kernel first. For a symmetric kernel such as a box or a Gaussian the two are identical, which is why it makes no difference here; for an asymmetric kernel the results would differ.

6. Task 2 — Gaussian filtering at 5x5 and 21x21

A Gaussian filter also averages, but the weights fall off with distance from the centre instead of being equal, which produces a far more natural blur.



Figure 7 — original, 5x5 Gaussian and 21x21 Gaussian.

Image	Standard deviation
Original	74.08
Gaussian 5x5	72.75
Gaussian 21x21	70.58

6.1 What the zero argument does — and a surprise

Passing `0` for `sigmaX` tells OpenCV to derive sigma from the kernel size, using sigma = 0.3((k-1)/2 - 1) + 0.8. For the two filters used here that gives **sigma = 1.1** and **sigma = 3.5**, so the two calls differ by more than their window size.

Passing that derived sigma back explicitly ought to reproduce the image exactly. It does not — the largest difference is 4 intensity levels. Printing the kernel OpenCV actually uses explains why:

```
cv2.getGaussianKernel(5, -1) -> [0.0625, 0.25, 0.375, 0.25, 0.0625]
multiplied by 16 -> [1. 4. 6. 4. 1.]
```

The weights are the **binomial** coefficients 1, 4, 6, 4, 1 over 16, not samples of a Gaussian curve. For small windows OpenCV substitutes this fixed integer kernel, which is a close approximation that is faster and exact in integer arithmetic. It corresponds to roughly sigma = 1.1, which is why the two results come within a few levels of each other rather than matching exactly.

6.2 Box versus Gaussian at the same size

Both kernels are 7x7 and both preserve brightness, but the box filter produces visibly harder, blockier edges because a distant neighbour carries the same weight as the centre pixel. This is the practical consequence of the weight profile in Figure 1.



Figure 8 — the same detail smoothed by a 7x7 box filter and a 7x7 Gaussian.

7. Task 3 — Sharpening with a 3x3 Laplacian

The Laplacian is a **second-derivative** operator. The first derivative is large wherever the intensity is changing; the second derivative is large wherever that rate of change itself changes — which is where edges and fine detail live. Its kernel sums to zero, so the output has a mean of -4.9e-05, essentially zero, and spans -6.235 to 3.6 only at the detail.

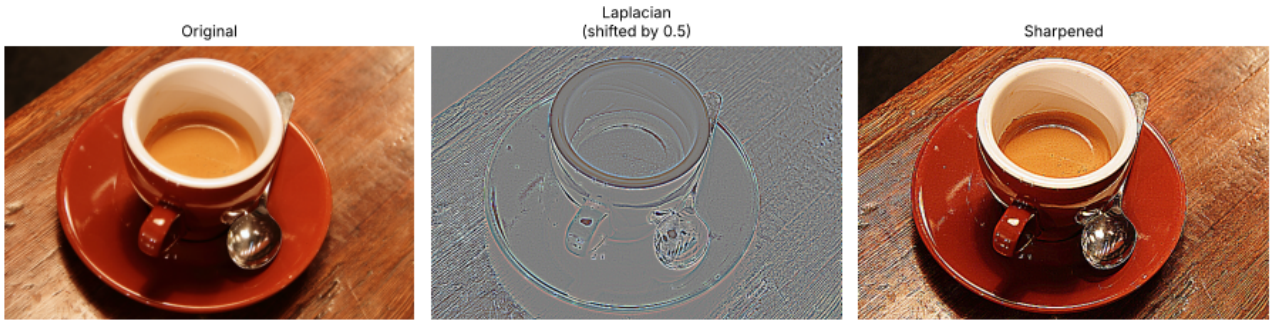


Figure 9 — original, the Laplacian response (shifted by 0.5 so negatives are visible), and the sharpened result.

7.1 Why the formula subtracts the Laplacian

OpenCV's Laplacian kernel has a **negative centre**: [[0,1,0],[1,-4,1],[0,1,0]]. At a bright point surrounded by darker pixels the output is therefore negative, and subtracting a negative value makes that point brighter. So `f - laplacian` raises peaks and deepens troughs, which is sharpening. Had the centre been positive, the correct formula would be addition.

Using the wrong sign raises no error, so the only way to catch it is to measure. The table below reports the mean value at the strongest 1% of peaks and troughs, identified by the Laplacian itself.

Version	At bright peaks	At dark troughs	Std. deviation
original	0.6892	0.2648	0.2905
f - laplacian	1.0000	0.0000	0.3424
f + laplacian	0.0000	1.0000	0.3234

The numbers settle it. With `f - laplacian` the bright points rise and the dark points fall — contrast at the detail increases. With `f + laplacian` the two **swap completely**: bright points go to black and dark points go to white, so every edge comes out inverted.

One observation is worth recording. The standard deviation **rises in both cases**, because both versions add energy at the edges. A common instinct is to treat a higher standard deviation as proof of sharpening, and here that instinct would accept the wrong formula. Only the direction of the change at peaks and troughs distinguishes them.



Figure 10 — the correct and the inverted result side by side.

7.2 The Laplacian amplifies noise

Because it responds to rapid change, the Laplacian cannot distinguish a genuine edge from a noisy pixel. Applied to a noisy image it amplifies the noise along with the detail. Blurring slightly first — the combination known as the Laplacian of Gaussian — is the standard remedy.

Version	Noise level (std of the error)
Noisy input	0.0387
Laplacian applied directly	0.3045
Blurred first, then Laplacian	0.0591

Applying the Laplacian directly multiplies the error by roughly 8 times, while pre-blurring keeps it close to the original noise level.



Figure 11 — a noisy image sharpened directly and with a Gaussian blur applied first.

7.3 Unsharp masking compared with the Laplacian

Both sharpen and both work by subtraction: unsharp masking subtracts a blurred copy, the Laplacian subtracts a second-derivative estimate of the same thing. The practical difference is control. Unsharp masking exposes `sigma` and `amount`, so the scale of the detail and the strength of the effect can be tuned separately, while the plain Laplacian has a fixed strength.



Figure 12 — the two sharpening methods on the same image.

8. Observations and conclusion

8.1 The filters at a glance

Filter	Weights	Sum	Effect
Box	all equal	1	plain average; blocky
Gaussian	taper with distance	1	weighted average; natural
Laplacian	negative centre, positive neighbours	0	detects change; sharpens

8.2 Points that required care

Issue	What happens	How it was handled
uint8 in sharpening	the mask is negative on half of every edge	compute in float32, clip once at the end
sigma = 0 in GaussianBlur	sigma is derived from the kernel size	computed it explicitly: 1.10 and 3.50
Small Gaussian kernels	OpenCV substitutes a binomial kernel	printed the kernel instead of assuming
Laplacian sign	the wrong sign inverts edges, silently	measured the value at peaks and troughs
Std. deviation as a check	it rises for both signs	used direction of change, not magnitude
Laplacian on noisy input	noise is amplified with the detail	blur first (Laplacian of Gaussian)

8.3 Conclusion

Spatial filtering is a single mechanism applied with different numbers. A kernel slides over the image and each output pixel becomes a weighted sum of its neighbourhood, so smoothing and sharpening are not separate algorithms but different weights. The sum of those weights predicts the behaviour: a kernel summing to 1 averages and preserves brightness, while a kernel summing to 0 returns nothing on flat regions and therefore detects change.

Sharpening in both forms works by subtraction. Unsharp masking isolates the detail a blur removed and adds it back amplified; the Laplacian estimates the same detail through a second derivative. The arithmetic must be done in float, because the quantity being added back is negative across one side of every edge.

The broader lesson from this lab is that several of the defaults behave differently from what their names suggest — a zero sigma is not 'no blur', a small Gaussian kernel is not actually Gaussian, and a rising standard deviation is not proof of sharpening. Each was settled by printing the value or measuring the effect rather than assuming it.

9. Resources

OpenCV — smoothing images
https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html

Data Carpentry — blurring images
https://datacarpentry.github.io/image-processing/06-blurring.html